CCCS 104 - Data Structures and Algorithms

LEARNING TASK (LINEAR DATA STRUCTURE - STACK)

NAME: **ORIAS, MARK JOSEPH C.** SECTION: ***2A***

RATIONALE

Explain briefly Stack Linear Data Structure, how it works? What are the common examples? Its applications?
Introduce your develop Python program, what can it do?

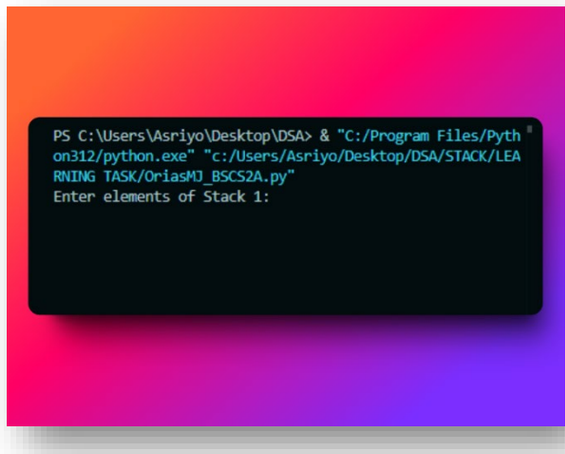
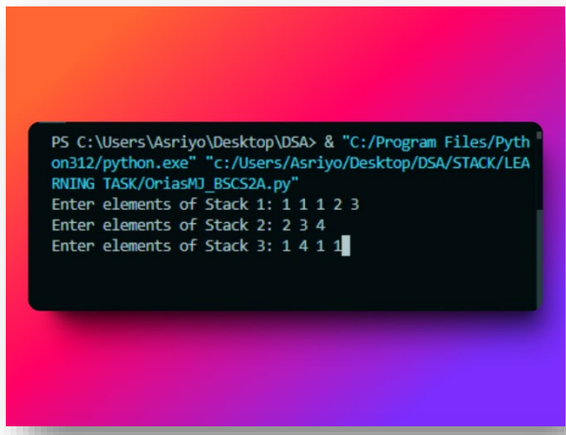
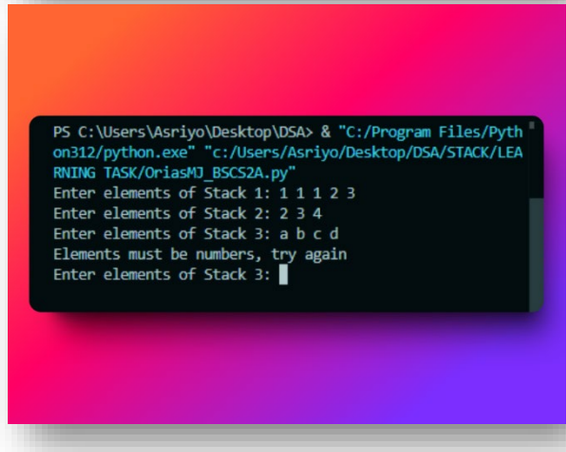
A ***stack*** is a fundamental linear data structure that operates based on the *Last In, First Out* (**LIFO**) principle, meaning that the last element added to the stack is the first one to be removed. A stack typically uses an array or linked list for implementation. When an item is *pushed*, it is placed at the top, and when *popped*, the most recently added item is removed. If you try to pop from an empty stack, it results in an "**underflow**" condition, while trying to push onto a full stack results in an "**overflow**."

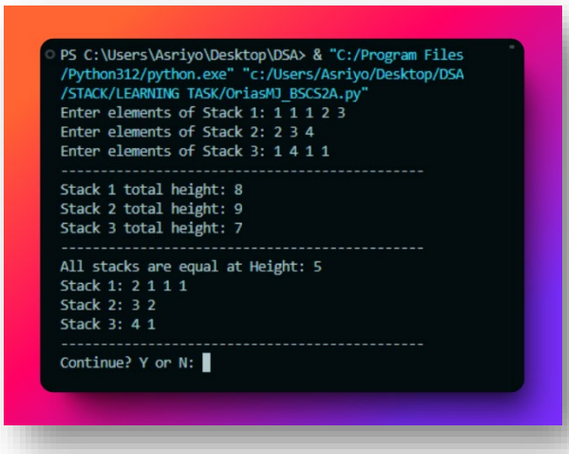
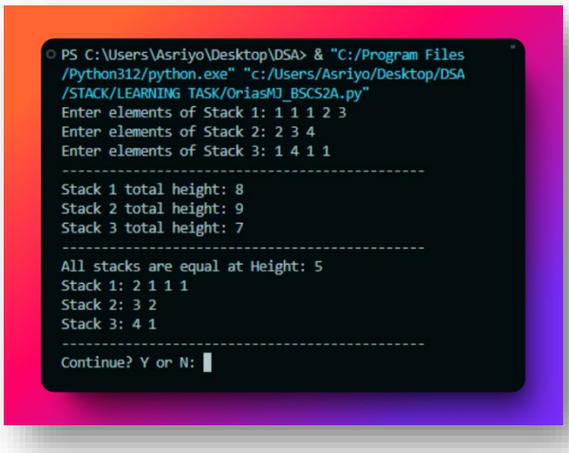
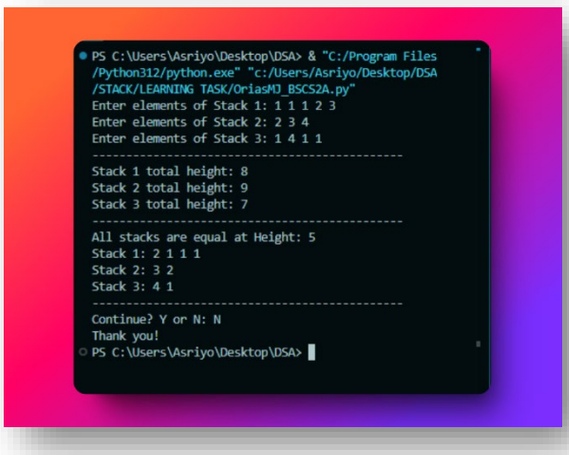
Some common examples of stacks in real life are a stack of plates, where you can only add or remove the plate at the top of the stack. Similarly, in a stack of books, you can only access the top book. Both scenarios demonstrate the stack data structure's "last in, first out" (LIFO) principle, where the most recently added item is the first to be removed. We can also see this being applied in ***backtracking algorithms***, like maze solving, ***undo operations*** in software text editors or drawing tools and ***browser history*** by going back to previous web pages.

In this learning task, I was tasked with developing a stack program implementing linked list. Unlike an array, a linked list stores data in a series of connected nodes, resembling a chain. Each node contains a reference to the next node, creating a linked structure. The program creates three stacks, gets elements for each stack from the user, pushes the elements to each stack, calculates and displays the total height of each stack, adjusts the stacks to have the same height, and displays the elements of each stack up to the equal height in LIFO order.

USER GUIDE

Step by step instructions on how to use your program. Include images for easily visualization

Step 1	*Image 1*
***RUN** the code until the program starts.* *Then, the user is asked to **ENTER** the elements of the stack.* *Elements of the stack must be separated by a " " space.*	
Step 2	*Image 2*
***ENTER** the elements you would like to insert in the stack.* *The user will be asked to input 3 times for the 3 stacks.* ***IF** the user enters an **INVALID** value, (e.g., "a", "$"), the program will print an error message and will ask again to input a number only.*	 

Step 3	*Image 3*
*After entering the values, press **"ENTER"** and the results will be displayed*	
Step 4	*Image 4*
***AFTER** the user is done using the program.* ***INPUT** "N" to **EXIT** the program and a "Thank you!' message will be displayed* ***IF** the user wants to continue using the program,* ***INPUT** "Y" and the program will ask again for the elements of the stack.*	 

PROGRAM CODE

Insert your Python code here, be sure to add comments to describe each line/set of codes…

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

#Linked list class
class LinkedList:
    def __init__(self):
        self.head = None

# Append data to the linked list
    def append(self, data):

        new_node = Node(data)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

# Remove the last element in the linked list
    def remove_last(self):

        if not self.head:
            return None
        if not self.head.next:
            data = self.head.data
            self.head = None
            return data
        current = self.head
        while current.next.next:
            current = current.next
        data = current.next.data
        current.next = None
        return data

# Display the linked list
    def display(self):
        elements = []
        current = self.head
        while current:
            elements.append(current.data)
            current = current.next
        return elements
```

```
# Calculate the total height represented by the linked list
    def calculate_height(self):
        current = self.head
        height = 0
        while current:
            height += current.data
            current = current.next
        return height

# Stack class using linked list as underlying storage
class Stack:
    def __init__(self):
        self.linked_list = LinkedList()

# Push an element onto the stack
    def push(self, data):
        self.linked_list.append(data)

# Pop an element from the stack
    def pop(self):
        return self.linked_list.remove_last()

# Get the total height of the stack
    def height(self):
        return self.linked_list.calculate_height()

# Display the stack
    def display(self):
        return self.linked_list.display()

# Display the stack in reverse order (LIFO)
    def display_reverse(self):
        return self.linked_list.display()[::-1]

def main():
    # Create three stacks
    stack1 = Stack()
    stack2 = Stack()
    stack3 = Stack()

    # Get elements for each stack
    while True:
        try:
            elements1 = list(map(int, input("Enter elements of Stack 1: ").split()))
            break
        except ValueError:
            print("Elements must be numbers, try again")
    while True:
        try:
            elements2 = list(map(int, input("Enter elements of Stack 2: ").split()))
            break
```

```
        except ValueError:
            print("Elements must be numbers, try again")
    while True:
        try:
            elements3 = list(map(int, input("Enter elements of Stack 3: ").split()))
            break
        except ValueError:
            print("Elements must be numbers, try again")

    # Push elements to each stack
    for element in elements1:
        stack1.push(element)
    for element in elements2:
        stack2.push(element)
    for element in elements3:
        stack3.push(element)

    # Calculate and display total height of each stack
    print("-----------------------------------------------")
    print("Stack 1 total height:", stack1.height())
    print("Stack 2 total height:", stack2.height())
    print("Stack 3 total height:", stack3.height())
    print("-----------------------------------------------")

    # Adjust all stacks to have the same height
    stacks = [stack1, stack2, stack3]
    heights = [stack.height() for stack in stacks]

    # Keep reducing the tallest stack until all heights are equal or all are zero
    while len(set(heights)) != 1:
        if all(height == 0 for height in heights):
            print("Stack heights will never be equal.")
            break
        max_height_index = heights.index(max(heights))
        stacks[max_height_index].pop()
        heights[max_height_index] = stacks[max_height_index].height()

    # All stacks should now have the same height
    equal_height = heights[0]

    # Display elements of each stack up to the equal height in LIFO order
    if equal_height == 0:
        print("Stack heights will never be equal.")
    else:
        print("All stacks are equal at Height:", equal_height)
        print("Stack 1:", *stack1.display_reverse())
        print("Stack 2:", *stack2.display_reverse())
        print("Stack 3:", *stack3.display_reverse())
    print("-----------------------------------------------")

    # Ask if user wants to continue
    while True:
```

```
        answer = input("Continue? Y or N: ").upper()
        if answer == "Y":
            main()
            break
        elif answer == "N":
            print("Thank you!")
            break
        else:
            print("Invalid input, please enter Y or N")

if __name__ == "__main__":
    main()
```

TUTORIAL VIDEO

YouTube Link: https://youtu.be/3uwwa7PHAMA

TAKEAWAYS

LEARNINGS:

In this learning task, I learned how to implement a stack using a linked list, which follows the Last-In-First-Out (LIFO) principle. In this implementation, each element in the stack is a node, with each node containing a value and a pointer to the next node. When I push an item, it gets added to the top of the stack, and during a pop, the top item is removed, with the pointers being updated accordingly. I also learned the following:

- That using classes and functions improves code structure by promoting reusability, reducing redundancy, and making complex logic easier to manage. This leads to cleaner, more scalable, and maintainable code.
- That combining loops with exception handling ensures programs can recover from invalid inputs and continue running smoothly. This approach enhances user experience by preventing crashes and allowing repeated input attempts.
- That comments are essential for maintaining clear, understandable code. They make complex logic easier to follow, aiding future development and collaboration.